

One To Many XML

(Advised to do it by your own)

One To Many Annotation

Case 1:

Step 1. Create the Persistent class

Query1N.java

```
package hibernatemapping;

import java.util.List;

import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.OneToMany;
import javax.persistence.OrderColumn;

@Entity
public class Query1N {
    @Id
    @Column(length = 10)
    private int id;
    @Column(length = 20)
    private String qname;
    @OneToMany(cascade = CascadeType.ALL)
    @JoinColumn(name="qid")
    @OrderColumn(name="type")
    private List<Response1N> responses;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getQname() {
        return qname;
    }
    public void setQname(String qname) {
        this.qname = qname;
    }
}
```

```

}
public List<Response1N> getResponses() {
    return responses;
}
public void setResponses(List<Response1N> responses) {
    this.responses = responses;
}
}

```

Response1N.java

```

package hibernatemapping;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;

@Entity
public class Response1N {
    @Id
    @Column(length = 10)
    private int id;

    @Column(length = 30)
    private String responsestr;

    @Column(length = 15)
    private String postedby;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getResponsestr() {
        return responsestr;
    }
    public void setResponsestr(String responsestr) {
        this.responsestr = responsestr;
    }
    public String getPostedby() {
        return postedby;
    }
    public void setPostedby(String postedby) {

```

```

        this.postedby = postedby;
    }
}

```

Step 2: Add project information and configuration in pom.xml file

pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>MavenProject</groupId>
    <artifactId>com.java</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <dependencies>
        <!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->
        <dependency>
            <groupId>org.hibernate</groupId>
            <artifactId>hibernate-core</artifactId>
            <version>4.1.6.Final</version>
        </dependency>

        <!-- https://mvnrepository.com/artifact/com.oracle.database.jdbc/ojdbc10 -->
        <dependency>
            <groupId>com.oracle.database.jdbc</groupId>
            <artifactId>ojdbc10</artifactId>
            <version>19.16.0.0</version>
        </dependency>
    </dependencies>
</project>

```

Step 3: Create the configuration file

hibernate.cfg.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property
name="hibernate.current_session_context_class">thread</property>

```

```

        <property
name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriv
er</property>
        <property
name="hibernate.connection.password">oracle</property>
        <property
name="hibernate.connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
roperty>
        <property
name="hibernate.connection.username">system</property>
        <property
name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</prope
rty>
        <property name="hbm2ddl.auto">create</property>
        <property name="show_sql">true</property>
        <mapping class="hibernatemapping.Query1N"/>
    </session-factory>
</hibernate-configuration>

```

Step 4: Create the class to store the data

AppQR1N.java

```

package hibernatemapping;

import java.util.ArrayList;
import java.util.List;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.service.ServiceRegistry;
import org.hibernate.service.ServiceRegistryBuilder;

public class AppQR1N {
    public static void main(String[] args) {

        Response1N r11 = new Response1N();
        r11.setId(11);
        r11.setResponsestr("Java is a programming Language");
        r11.setPostedby("Joe");

        Response1N r12 = new Response1N();
        r12.setId(12);
        r12.setResponsestr("Java is a programming Language");
    }
}

```

```

r12.setPostedby("Joseph");

List<Response1N> list1=new ArrayList<Response1N>();
list1.add(r11);
list1.add(r12);

Response1N r21 = new Response1N();
r21.setId(21);
r21.setResponsestr("Servlet is an Interface");
r21.setPostedby("Jas");

Response1N r22 = new Response1N();
r22.setId(22);
r22.setResponsestr("Servlet is J2EE Concept");
r22.setPostedby("Rohan");

List<Response1N> list2=new ArrayList<Response1N>();
list2.add(r21);
list2.add(r22);

Query1N query1 = new Query1N();
query1.setId(101);
query1.setQname("What is JAVA?");
query1.setResponses(list1);

Query1N query2 = new Query1N();
query2.setId(102);
query2.setQname("What is SERVLET?");
query2.setResponses(list2);

Configuration conf = new Configuration()
    .configure("hibernate.cfg.xml")
    .addAnnotatedClass(Query1N.class)
    .addAnnotatedClass(Response1N.class);

ServiceRegistry sr = new ServiceRegistryBuilder()
    .applySettings(conf.getProperties())
    .buildServiceRegistry();
SessionFactory sf1 = conf.buildSessionFactory(sr);
Session s1 = sf1.openSession();
Transaction txn1 = s1.beginTransaction();

s1.save(query1);

```

```

s1.save(query2);

txn1.commit();
}
}

```

The screenshot shows the Eclipse IDE with a Java application running. The console output displays the following logs:

```

<terminated> AppQRIN [Java Application] C:\Users\HP\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_18.0.1.v20220515-1614\jre\bin\java.exe (18-Jul-2023, 1:08:34 pm - 1:08:43 pm) [pid: 15376]
Jul 18, 2023 1:08:42 PM org.hibernate.tool.hbm2ddl.SchemaExport perform
ERROR: ORA-00942: table or view does not exist

Hibernate: drop table Response1N cascade constraints
Jul 18, 2023 1:08:42 PM org.hibernate.tool.hbm2ddl.SchemaExport perform
ERROR: HHH000389: Unsuccessful: drop table Response1N cascade constraints
Jul 18, 2023 1:08:42 PM org.hibernate.tool.hbm2ddl.SchemaExport perform
ERROR: ORA-00942: table or view does not exist

Hibernate: create table Query1N (id number(10,0) not null, qname varchar2(20 char), primary key (id))
Hibernate: create table Response1N (id number(10,0) not null, postedby varchar2(15 char), responsestr varchar2(30 char), qid number(10,0), type
Hibernate: alter table Response1N add constraint FK5114EA5E46B94E7F foreign key (qid) references Query1N
Jul 18, 2023 1:08:43 PM org.hibernate.tool.hbm2ddl.SchemaExport execute
INFO: HHH000230: Schema export complete
Hibernate: select response1n_.id, response1n_.postedby as postedby1_, response1n_.responsestr as response3_1_ from Response1N response1n_ where
Hibernate: select response1n_.id, response1n_.postedby as postedby1_, response1n_.responsestr as response3_1_ from Response1N response1n_ where
Hibernate: select response1n_.id, response1n_.postedby as postedby1_, response1n_.responsestr as response3_1_ from Response1N response1n_ where
Hibernate: insert into Query1N (qname, id) values (?, ?)
Hibernate: insert into Response1N (postedby, responsestr, id) values (?, ?, ?)
Hibernate: insert into Response1N (postedby, responsestr, id) values (?, ?, ?)
Hibernate: insert into Query1N (qname, id) values (?, ?)
Hibernate: insert into Response1N (postedby, responsestr, id) values (?, ?, ?)
Hibernate: insert into Response1N (postedby, responsestr, id) values (?, ?, ?)
Hibernate: update Response1N set qid=?, type=? where id=?
Hibernate: update Response1N set qid=?, type=? where id=?
Hibernate: update Response1N set qid=?, type=? where id=?

```

Below the console, a SQL Plus window shows the results of two queries:

```

SQL> select * from Query1N;

  ID QNAME
-----
 101 What is JAVA?
 102 What is SERVLET?

SQL> select * from Response1N;

  ID POSTEDBY      RESPONSESTR                                QID    TYPE
-----
 11 Joe           Java is a programming Language              101     0
 12 Joseph        Java is a programming Language              101     1
 21 Jas           Servlet is an Interface                     102     0
 22 Rohan         Servlet is J2EE Concept                     102     1

SQL>

```

Step 5: Create the class to fetch the data

AppQR1NFetch.java

```
package hibernatemapping;

import java.util.List;

import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import org.hibernate.service.ServiceRegistry;
import org.hibernate.service.ServiceRegistryBuilder;

public class AppQR1NFetch {
    public static void main(String[] args) {
        Configuration conf = new
        Configuration().configure("hibernate.cfg.xml")
            .addAnnotatedClass(Query1N.class)
            .addAnnotatedClass(Response1N.class);

        ServiceRegistry sr = new ServiceRegistryBuilder()
            .applySettings(conf.getProperties())
            .buildServiceRegistry();
        SessionFactory sf1 = conf.buildSessionFactory(sr);
        Session s1 = sf1.openSession();
        s1.beginTransaction();

        Query query = s1.createQuery("from Query1N");

        List<Query1N> list = query.list();

        for(Query1N q : list) {
            System.out.println("Query::");
            System.out.println(q.getId());
            System.out.println(q.getQname());

            List<Response1N> listres = q.getResponsees();

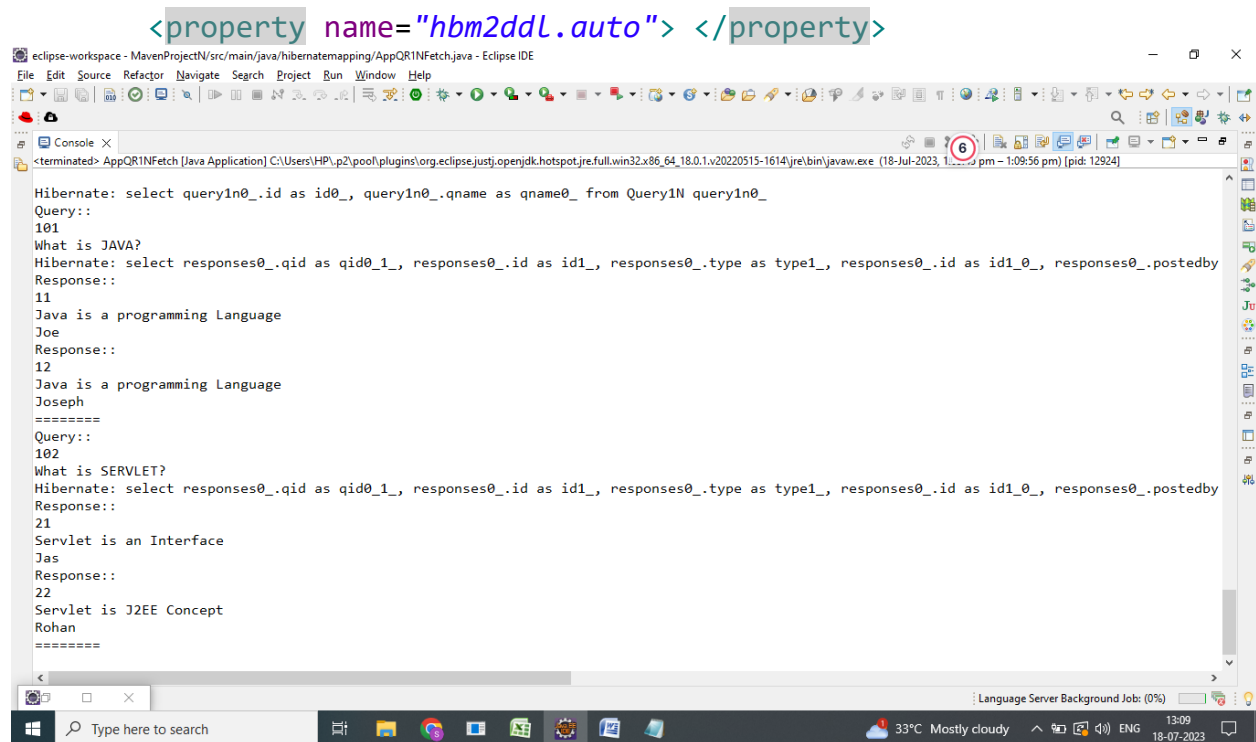
            for(Response1N res : listres) {
                System.out.println("Response::");
                System.out.println(res.getId());
                System.out.println(res.getResponseestr());
                System.out.println(res.getPostedby());
            }
        }
    }
}
```

```

System.out.println("=====");
}
s1.getTransaction().commit();
}
}

```

In **hibernate.cfg.xml** file, ensure to update



Case 2:

Query1N.java

```
package hibernatemapping;
```

```
import java.util.List;
```

```
import javax.persistence.CascadeType;
```

```
import javax.persistence.Column;
```

```
import javax.persistence.Entity;
```

```
import javax.persistence.Id;
```

```
import javax.persistence.OneToMany;
```

```
@Entity
```

```
public class Query1N {
```

```
    @Id
```

```
    @Column(length = 10)
```

```

private int id;
    @Column(length = 20)
private String qname;
@OneToMany(cascade = CascadeType.ALL)
private List<Response1N> responses;
public int getId() {
    return id;
}
public void setId(int id) {
    this.id = id;
}
public String getQname() {
    return qname;
}
public void setQname(String qname) {
    this.qname = qname;
}
public List<Response1N> getResponses() {
    return responses;
}
public void setResponses(List<Response1N> responses) {
    this.responses = responses;
}
}

```

The screenshot shows the Eclipse IDE interface. The top part displays the Java code for the `AppQR1N` class, which is a JPA entity with a primary key `id`, a `qname` attribute, and a `responses` collection. The bottom part shows the console output, which contains Hibernate SQL statements and messages from the hbm2ddl plugin. The console output includes the following text:

```

eclipse-workspace - MavenProjectN/src/main/java/hibernatemapping/AppQR1N.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
<terminated> AppQR1N [Java Application] C:\Users\HP\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_18.0.1.v20220515-1614\jre\bin\javaw.exe (18-Jul-2023, 1:00:32 pm - 1:00:42 pm) [pid: 9200]
Hibernate: drop table Response1N cascade constraints
Hibernate: create table Query1N (id number(10,0) not null, qname varchar2(20 char), primary key (id))
Hibernate: create table Query1N_Response1N (Query1N_id number(10,0) not null, responses_id number(10,0) not null, responsestr varchar2(15 char), responsestr_id number(10,0) not null, primary key (Query1N_id, responses_id))
Hibernate: create table Response1N (id number(10,0) not null, postedby varchar2(15 char), responsestr varchar2(15 char), responsestr_id number(10,0) not null, primary key (id, responsestr_id))
Hibernate: alter table Query1N_Response1N add constraint FK650909F83A26AFA8 foreign key (Query1N_id) references Query1N (id)
Hibernate: alter table Query1N_Response1N add constraint FK650909F8AC47AF78 foreign key (responses_id) references Response1N (id)
Jul 18, 2023 1:00:41 PM org.hibernate.tool.hbm2ddl.SchemaExport execute
INFO: HHH000230: Schema export complete
Hibernate: select response1n_.id, response1n_.postedby as postedby1_, response1n_.responsestr as response3_1, response1n_.responsestr_id as response3_1_id from Response1N response1n_
Hibernate: select response1n_.id, response1n_.postedby as postedby1_, response1n_.responsestr as response3_1, response1n_.responsestr_id as response3_1_id from Response1N response1n_
Hibernate: select response1n_.id, response1n_.postedby as postedby1_, response1n_.responsestr as response3_1, response1n_.responsestr_id as response3_1_id from Response1N response1n_
Hibernate: insert into Query1N (qname, id) values (?, ?)
Hibernate: insert into Response1N (postedby, responsestr, id) values (?, ?, ?)
Hibernate: insert into Response1N (postedby, responsestr, id) values (?, ?, ?)
Hibernate: insert into Query1N_Response1N (Query1N_id, responses_id) values (?, ?)
Hibernate: insert into Query1N_Response1N (Query1N_id, responses_id) values (?, ?)
Hibernate: insert into Query1N_Response1N (Query1N_id, responses_id) values (?, ?)

```

```
SQL Plus

SQL> select * from Query1N;

      ID QNAME
-----
    101 What is JAVA?
    102 What is SERVLET?

SQL> select * from Response1N;

      ID POSTEDBY      RESPONSESTR
-----
    11 Joe          Java is a programming Language
    12 Joseph        Java is a programming Language
    21 Jas            Servlet is an Interface
    22 Rohan          Servlet is J2EE Concept

SQL> select * from Query1N_Response1N;

QUERY1N_ID RESPONSES_ID
-----
    101          11
    101          12
    102          21
    102          22

SQL>
```

Step 5: Create the class to fetch the data (Same as above).

In **hibernate.cfg.xml** file, ensure to update

```
<property name="hbm2ddl.auto"> </property>
```

```
eclipse-workspace - MavenProjectN/src/main/java/hibernatemapping/AppQR1NFetch.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

<terminated> AppQR1NFetch [Java Application] C:\Users\HP\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.18.0.1.x20220515-1614\jre\bin\javaw.exe (18-Jul-2023, 1:03:48 pm - 1:04:00 pm) [pid: 9656]

Hibernate: select query1n0_.id as id0_, query1n0_.qname as qname0_ from Query1N query1n0_
Query::
101
What is JAVA?
Hibernate: select responses0_.Query1n_id as Query1_0_1_, responses0_.responses_id as responses2_1_, response1n1_.id as id1_0_, response1n1_.post
Response::
11
Java is a programming Language
Joe
Response::
12
Java is a programming Language
Joseph
=====
Query::
102
What is SERVLET?
Hibernate: select responses0_.Query1n_id as Query1_0_1_, responses0_.responses_id as responses2_1_, response1n1_.id as id1_0_, response1n1_.post
Response::
21
Servlet is an Interface
Jas
Response::
22
Servlet is J2EE Concept
Rohan
=====

Language Server Background Job: (0%)
```

